ADG POWER OS

Engineering specification and implementation handoff

Lightweight Ubuntu developer environment | MSI Cyborg 15 B13WFKG | Version 1.0 | 10 November 2026





1. Executive summary

ADG Power OS is a selectable, reversible low-power Ubuntu session, not a separate operating system. It provides a modern Tailwind-style developer desktop, live battery telemetry, Wi-Fi and Bluetooth management, Claude Code, on-demand Chrome, and minimal background services. The target is 5–15 W at the battery under defined light workloads; this is an engineering aspiration, not a guaranteed result.

2. Success criteria and power budgets

Profile	Target	Expected behavior
Deep Eco	5–8 W	Terminal, Wi-Fi idle, dim/blank display; aspirational
Developer	8–12 W	Claude Code, terminal, live telemetry
Connected	10–15 W	Wi-Fi, Bluetooth audio, light browsing
Performance	15 W+	Builds, heavier browsing, GPU tasks

3. Core user experience

A branded Sway/Wayland session with a Tauri + React + Tailwind dashboard. Show battery percentage, discharge watts, smoothed history, estimated runtime, charge status, CPU/RAM utilization, RTX runtime state, current Wi-Fi SSID, Bluetooth audio state, and power profile. A left dock launches terminal/Claude, Chrome, files, Wi-Fi, Bluetooth, music, settings. Use the provided design tokens and mockups as visual references, not pixel-perfect requirements.

4. Boot architecture and reversibility

GRUB offers Ubuntu Normal and ADG Power OS. Low-power selection passes an identifiable kernel argument or boots a dedicated systemd target; an early, testable policy applies the low-power profile. Do not modify the normal boot entry. Preserve recovery entry and document rollback. Prefer a separate user session over system-wide changes until proven safe. Avoid brittle hard-coded kernel paths.

5. Service policy

In eco mode do not start Docker, Docker socket, containerd, Kubernetes/k3s, VMs, database services, developer servers, auto-indexers or update jobs unless explicitly requested. Account for system and user services, sockets and timers; stop only when safe. NetworkManager remains enabled. Bluetooth and PipeWire are on demand. Normal Ubuntu must retain original service behavior.

6. Power-management policy

Use powerprofilesctl power-saver when supported. Set keyboard illumination off, reduce brightness, prefer 60Hz if available, and enable Wi-Fi power saving with a latency-sensitive override. Attempt NVIDIA runtime suspend using supported driver mechanisms; verify via sysfs before displaying Suspended. Never forcibly unbind the GPU or disable critical drivers. Provide per-setting revert controls.

7. Telemetry and runtime estimation

Auto-discover BAT* directories. When power_now exists, divide microW by 1e6; otherwise compute current_now (microA) times voltage_now (microV) divided by 1e12. Remaining energy: prefer energy_now/1e6 Wh; otherwise

charge_now (microAh) times voltage_now (microV) /1e12 Wh. Runtime hours = remaining Wh / smoothed discharge W. Distinguish AC charging, battery charging, full, unknown, and discharging; hide runtime when not meaningful. Store bounded history; avoid high-frequency polling.

8. Network and Bluetooth

NetworkManager DBus integration: scan SSIDs, show signal/security, connect/disconnect, enter credentials securely, show IP and current network. BlueZ DBus integration: discover, pair, trust, connect/disconnect devices. PipeWire/WirePlumber: choose output device and show playback status. Wi-Fi must work without opening the GUI.

9. App launching and media

Launch Claude Code in foot/kitty, Chrome on demand, file manager and settings. Music through Chrome and Bluetooth headphones. Keep the dashboard lightweight and allow exiting it without stopping the daemon or networking. Do not promise a 5–15W ceiling when browser, media or builds are active.

10. System architecture

Rust telemetry/control service with least privilege; D-Bus and sysfs adapters; Tauri UI; Sway session; NetworkManager, BlueZ, PipeWire; systemd boot policy. Privileged operations must use explicit polkit/systemd authorization and allowlisted actions. No arbitrary shell execution from untrusted UI values.

11. Security and safety

Never expose Wi-Fi passwords in logs; do not run UI as root. Provide clear prompts before altering service states, sleep policies or boot configuration. Protect normal boot and data. Maintain audit logs of profile changes. Battery measurements are approximate and may vary by charging status and sensor semantics.

12. Acceptance tests

A: normal Ubuntu boots unchanged and Docker still works. B: eco boot starts no containers, docker sockets, k3s, databases or unnecessary daemons. C: Wi-Fi scan/connect survives UI close. D: Bluetooth pairing and Chrome audio work. E: correct battery math with BAT1 and missing sensor fields. F: runtime estimates disappear on charging/zero draw. G: RTX suspension indicator matches kernel state. H: return to normal boot and rollback documented. I: record five-minute stabilized power averages across profiles.

13. Build phases for Claude Code

Phase 0 hardware discovery and baseline; Phase 1 Rust telemetry CLI and tests; Phase 2 Tauri/React dashboard; Phase 3 Wi-Fi/Bluetooth/audio; Phase 4 Sway session and application launcher; Phase 5 reversible boot profile; Phase 6 performance tuning and QA. Deliver install/uninstall scripts, tests, screenshots, a service matrix and reproducible benchmark report.

14. Deliverables

Source code for Tauri frontend and Rust daemon, Linux service units and boot integration, wallpaper assets, SVG brand mark, design tokens, test fixtures, deployment guide, rollback guide, and measured power report. No claims of reaching 5W without actual measurements.

Visual assets and wallpaper

Included are the original desktop mockup, an overview board, 4K and 1440p wallpapers, a scalable SVG logo and Tailwind-compatible design tokens.



Wallpaper is a separate original graphic asset; no interface text is baked into it.